

230. Kth Smallest Element in a BST Medium

Acceptance: 76.3% | [LeetCode](#)

Given the `root` of a binary search tree, and an integer `k`, return the `kth` smallest value (**1-indexed**) of all the values of the nodes in the tree.

Example 1:

```
**Input:** root = [3,1,4,null,2], k = 1
**Output:** 1
```

Example 2:

```
**Input:** root = [5,3,6,2,4,null,null,1], k = 3
**Output:** 3
```

Constraints:

- The number of nodes in the tree is `n`.
- `1 <= k <= n <= 104`
- `0 <= Node.val <= 104`

Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the `kth` smallest frequently, how would you optimize?

Claude Opus Solution

Kth Smallest Element in a BST

1. Intuition

The **key insight** is a fundamental property of Binary Search Trees:

An in-order traversal of a BST visits nodes in sorted (ascending) order.

This is because in-order traversal follows the pattern: `left → node → right`. Since in a BST all left-subtree values `< node value < all right-subtree values`, processing nodes in this order naturally yields them sorted.

So the problem reduces to: **"Find the `kth` element of a sorted sequence."** We simply perform in-order traversal and stop when we've visited `k` nodes—no need to traverse the entire tree.

2. Approach (Iterative In-Order Traversal)

We use an explicit stack to simulate in-order traversal so we can stop early.

1. Initialize an empty stack.
2. Loop:
 - a. Push all left-children onto stack (go as far left as possible).
This stages the smallest unvisited nodes.
 - b. Pop a node from the stack — this is the next smallest value.
 - c. Decrement `k`. If `k == 0`, this node is our answer; return it.
 - d. Move to the popped node's right child (to process the right subtree), then repeat from step (a).

The pattern: **"go left as deep as possible, process node, then go right."**

3. Python Solution

```
from typing import Optional

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        # Stack holds nodes whose left subtrees we've already pushed,
        # but which we haven't "visited" (processed) yet.
```

```

stack = []

while True:
    # Step 1: Dive as far LEFT as possible.
    # Each pushed node is a candidate; the deepest-left node
    # we reach is the smallest unprocessed value.
    while root:
        stack.append(root)    # remember this node to process later
        root = root.left     # keep going left toward smaller values

    # Step 2: Pop the smallest unprocessed node.
    # Because we drained all left children, the top of the stack
    # is the next node in sorted (in-order) sequence.
    root = stack.pop()

    # Step 3: We've now "visited" one more node in sorted order.
    k -= 1
    if k == 0:
        # This is the kth smallest – return immediately (early exit).
        return root.val

    # Step 4: Move to the RIGHT subtree.
    # Everything in the right subtree is larger than the current node
    # but smaller than ancestors still on the stack – correct order.
    root = root.right

```

4. Complexity Analysis

Time Complexity: $O(H + k)$ - H = height of the tree. We first descend down the leftmost path (up to H pushes) before processing our first node. - We then process k nodes, each involving $O(1)$ amortized work (each node is pushed and popped exactly once). - In the worst case $k = n$ and $H = n$ (skewed tree), giving **$O(n)$** .

Space Complexity: $O(H)$ - The stack holds at most H nodes (the current root-to-leftmost path). - Balanced tree: **$O(\log n)$** . Skewed tree: **$O(n)$** .

Note: The recursive solution has the same complexity, but uses the implicit call stack instead of an explicit one.

5. Alternative Approaches

A) Recursive In-Order with Early Stop

Clean and intuitive, but harder to short-circuit cleanly (must thread state through recursion):

```

class Solution:
    def kthSmallest(self, root, k):
        self.k = k
        self.result = None

        def inorder(node):
            if not node or self.result is not None:
                return
            inorder(node.left)
            self.k -= 1
            if self.k == 0:
                self.result = node.val
                return
            inorder(node.right)

        inorder(root)
        return self.result

```

- Time: $O(H + k)$, Space: $O(H)$ (call stack).

B) Full In-Order into a List

Traverse everything into a sorted list, then index `arr[k-1]`. - Simple but always $O(n)$ time/space; no early exit. Wasteful for small k .

Follow-up: Frequent Inserts/Deletes

If the BST is modified often and we query the k th smallest frequently:

Augment each node with a `count` field = size of its subtree (number of nodes rooted there). Then we can find the k th smallest in **$O(H)$** time:

```
def kthSmallest(node, k):
    left_size = count(node.left)    # nodes smaller than node
    if k == left_size + 1:
        return node.val             # current node is the kth
    elif k <= left_size:
        return kthSmallest(node.left, k)    # search left
    else:
        return kthSmallest(node.right, k - left_size - 1) # search right
```

- **Query:** $O(H)$ instead of $O(k)$.
- Insert/delete: still $O(H)$, just update `count` fields along the path.
- With a **self-balancing BST** (AVL / Red-Black), $H = O(\log n)$, so all operations become **$O(\log n)$** — ideal for a dynamic, frequently-modified structure.

Codex Solution

1. Intuition

A **Binary Search Tree** has this important property:

- All values in the left subtree are smaller than the current node.
- All values in the right subtree are larger than the current node.

Therefore, if we do an **inorder traversal**:

```
left subtree -> current node -> right subtree
```

we visit the nodes in **sorted ascending order**.

So the problem becomes:

Perform inorder traversal and return the value of the k th visited node.

For example:

```
BST inorder traversal = [1, 2, 3, 4, 5, 6]
k = 3
answer = 3
```

The key optimization is that we do **not** need to traverse the entire tree.

We can stop as soon as we visit the k th node.

2. Approach

We can implement inorder traversal iteratively using a stack.

Pseudocode

```
function kthSmallest(root, k):
    stack = empty stack
    current = root

    while current is not null OR stack is not empty:

        while current is not null:
            push current into stack
            current = current.left

        current = pop from stack

        k = k - 1
        if k == 0:
            return current.val
```

```
current = current.right
```

Why this works

The stack simulates recursion.

For every node, we first go as far left as possible.

Then we process the node.

Then we move to its right subtree.

That is exactly inorder traversal.

3. Python Solution

```
from typing import Optional

# Definition for a binary tree node.
# LeetCode provides this class, so usually you do not need to redefine it.
class TreeNode:
    def __init__(self, val: int = 0, left: Optional["TreeNode"] = None, right: Optional["TreeNode"] = None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        # Stack used to simulate recursive inorder traversal
        stack = []

        # Start from the root of the BST
        current = root

        # Continue while there are nodes to process
        while current is not None or stack:

            # Step 1:
            # Go as far left as possible.
            # In a BST, smaller values are always on the left.
            while current is not None:
                stack.append(current)
                current = current.left

            # Step 2:
            # The top of the stack is now the next smallest unvisited node.
            current = stack.pop()

            # We have visited one more node in sorted order.
            k -= 1

            # If this is the kth visited node, return its value.
            if k == 0:
                return current.val

            # Step 3:
            # Move to the right subtree.
            # Those values are larger than the current node.
            current = current.right
```

4. Complexity Analysis

Let:

- n be the number of nodes in the tree
- h be the height of the tree

Time Complexity

```
O(h + k)
```

We first may go down the height of the tree, then we visit nodes until the k th smallest one.

In the worst case, $k = n$, so:

```
O(n)
```

Space Complexity

```
O(h)
```

The stack stores at most one path from the root to a leaf.

In the worst case, if the tree is completely skewed:

```
O(n)
```

If the tree is balanced:

```
O(log n)
```

5. Alternative Approaches

1. Recursive Inorder Traversal

We can also use recursion:

```
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        self.k = k
        self.answer = None

        def inorder(node):
            if node is None:
                return

            # Visit left subtree first
            inorder(node.left)

            # Process current node
            self.k -= 1
            if self.k == 0:
                self.answer = node.val
                return

            # Visit right subtree
            inorder(node.right)

        inorder(root)
        return self.answer
```

However, recursive traversal uses the function call stack, so its space complexity is also $O(h)$.

One issue is that this version does not perfectly stop all recursion after finding the answer unless we add extra early-return checks.

2. Store Full Inorder Traversal in a List

We can collect all values using inorder traversal:

```
values = []
inorder(root)
return values[k - 1]
```

This is simple, but less space-efficient.

Complexity:

```
Time: O(n)
Space: O(n)
```

3. Follow-up: Frequent Insert/Delete and Frequent kth Smallest Queries

If the BST is modified often and we need to find the kth smallest frequently, we can augment each node with its subtree size.

Each node stores:

```
node.size = number of nodes in subtree rooted at node
```

Then for a node:

```
left_size = size of node.left
```

Now:

- If $k \leq \text{left_size}$, the kth smallest is in the left subtree.
- If $k == \text{left_size} + 1$, the current node is the answer.
- If $k > \text{left_size} + 1$, search the right subtree with:

```
k = k - left_size - 1
```

This gives:

```
kthSmallest: O(h)
insert: O(h)
delete: O(h)
```

If the tree is self-balancing, such as an AVL tree, Red-Black tree, or Treap, then:

```
h = O(log n)
```

So all operations become:

```
O(log n)
```

This data structure is often called an **Order Statistic Tree**.